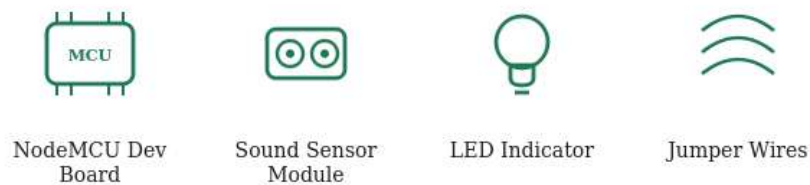




SOUND LEVEL MONITOR

*Real-Time Sound Level Monitoring and Alert System using NodeMCU (ESP8266)
Project Documentation*

Project by: M Jayantha Siva Srinivas



1. Abstract

This project presents the design and implementation of a real-time sound level monitoring and alert system built around the NodeMCU ESP8266 development board and an analog sound sensor module. The system continuously samples ambient sound, converts the measured signal into an approximate decibel (dB) value, and compares it against a configurable threshold. When the ambient sound level crosses this threshold, an onboard LED is triggered as a visual alert, and the live readings are streamed to a serial monitor for observation and logging. The project demonstrates core embedded systems concepts including analog signal acquisition, real-time signal processing, threshold-based decision logic, and digital output control — forming a practical foundation for more advanced IoT-based noise monitoring applications.

2. Objective

The primary objective of this project is to build a low-cost, standalone embedded system capable of measuring ambient sound intensity in real time and generating an immediate visual alert when the sound level exceeds a defined safe threshold. The specific goals are:

1. To interface an analog sound sensor with the NodeMCU ESP8266 microcontroller.
2. To sample the raw analog signal and derive a meaningful, approximate decibel (dB) reading from it.
3. To implement a threshold-based comparison logic that distinguishes between normal and excessive sound levels.
4. To provide an instantaneous visual indication (LED) when the sound level crosses the set threshold.
5. To continuously report the measured values over the serial interface for monitoring and debugging.
6. To lay the groundwork for future extension into a connected, Wi-Fi-enabled IoT noise-monitoring node.

3. Introduction

Noise pollution has become an increasingly relevant concern in both urban and industrial environments, affecting concentration, sleep quality, and long-term health. Embedded and IoT-based sensing systems offer an efficient and inexpensive way to monitor sound levels continuously and respond to unsafe conditions automatically, without relying on manual observation.

This project uses the NodeMCU ESP8266, a compact and Wi-Fi-capable microcontroller board widely used in IoT prototyping, as the processing unit. An analog sound sensor module, built around a small electret microphone and an onboard amplifier/comparator circuit, is connected to one of the board's analog input pins. The microcontroller repeatedly samples this analog signal over a short time window, calculates the peak-to-peak amplitude of the waveform, and converts this amplitude into an approximate sound level expressed in decibels.

Based on this calculated value, the system compares the current sound level against a preset threshold. If the level exceeds the threshold, an LED connected to a digital output pin is switched on to serve as an immediate, human-visible alert. All readings are simultaneously printed to the serial monitor, allowing the behaviour of the system to be observed and verified during development and testing.

While the current implementation operates as a standalone unit with local LED-based alerting, the choice of the ESP8266 as the controller was deliberate — its built-in Wi-Fi module allows the same hardware to be extended in future work to transmit sound data to a cloud dashboard, a mobile app, or a home-automation system, without any change to the sensing hardware.

4. Components Used

The system is built using a small set of readily available components, summarised below:

Component	Description / Role in the Project
NodeMCU ESP8266	Wi-Fi-enabled microcontroller board; performs sampling, dB calculation, threshold logic, and serial output
Analog Sound Sensor Module	Electret microphone with onboard amplifier; outputs an analog voltage proportional to ambient sound intensity
LED (Indicator)	Connected to digital pin D1; provides a visual alert when the sound threshold is exceeded
Jumper Wires	Used to connect the sensor module and LED to the NodeMCU's analog and digital pins
Micro-USB Cable	Provides power to the NodeMCU and enables serial communication with the computer
Laptop / PC with Arduino IDE	Used for writing, uploading the firmware, and viewing live readings via the Serial Monitor

5. System Architecture and Working Principle

The system follows a straightforward sense–process–act architecture, typical of embedded sensing applications:

5.1 Sensing Stage

The sound sensor's microphone picks up ambient acoustic vibrations and converts them into a small analog electrical signal. The sensor's onboard amplifier boosts this signal to a usable voltage range, which is then fed into the analog input pin (A0) of the NodeMCU.

5.2 Sampling and Signal Processing

Because sound is a rapidly varying waveform, a single instantaneous reading is not meaningful. The microcontroller therefore samples the analog input repeatedly over a short time window (50 milliseconds), tracking the highest and lowest values observed during that window. The difference between these two values — the peak-to-peak amplitude — represents the intensity of the sound captured during that window.

5.3 Decibel Approximation

The peak-to-peak amplitude, which is in raw ADC units, is converted into a voltage value based on the NodeMCU's analog reference range. This voltage is then passed through a logarithmic relationship (the standard basis for decibel calculations) to arrive at an approximate sound level in decibels. This is a relative, empirically-scaled measurement suited for threshold comparison rather than a laboratory-calibrated absolute dB(SPL) reading.

5.4 Threshold Comparison and Alerting

The calculated dB value is compared against a preset threshold value. If the measured level exceeds this threshold, the microcontroller drives the LED output pin HIGH, switching the LED on as an immediate alert. If the level is within the acceptable range, the LED remains off. This comparison and the corresponding action are repeated continuously in the main control loop.

5.5 Reporting

In parallel with the alerting logic, every computed sound level is transmitted over the serial interface at a fixed baud rate, allowing it to be viewed live on the Serial Monitor. This provides a continuous, human-readable log of the system's behaviour, which is useful both for testing and for future integration with data-logging or cloud-reporting features.

6. Hardware Setup

The physical assembly connects the sound sensor module and the LED to the NodeMCU's analog and digital pins as described below. The sound sensor's analog output is wired to pin A0, while the LED's control line is wired to digital pin D1, with the remaining connections providing power and ground to both peripherals.

Pin on NodeMCU	Connected To
A0 (Analog In)	Analog output (AO) of the sound sensor module
D1 (Digital Out)	Anode (+) of the alert LED, through a current-limiting arrangement
3V3 / VU	VCC of the sound sensor module
GND	GND of the sound sensor module and cathode side of the LED circuit

The photograph below shows the assembled hardware during a live test session, with the NodeMCU connected to the sound sensor and LED, and the Arduino IDE Serial Monitor on the laptop displaying real-time sound level readings in decibels.

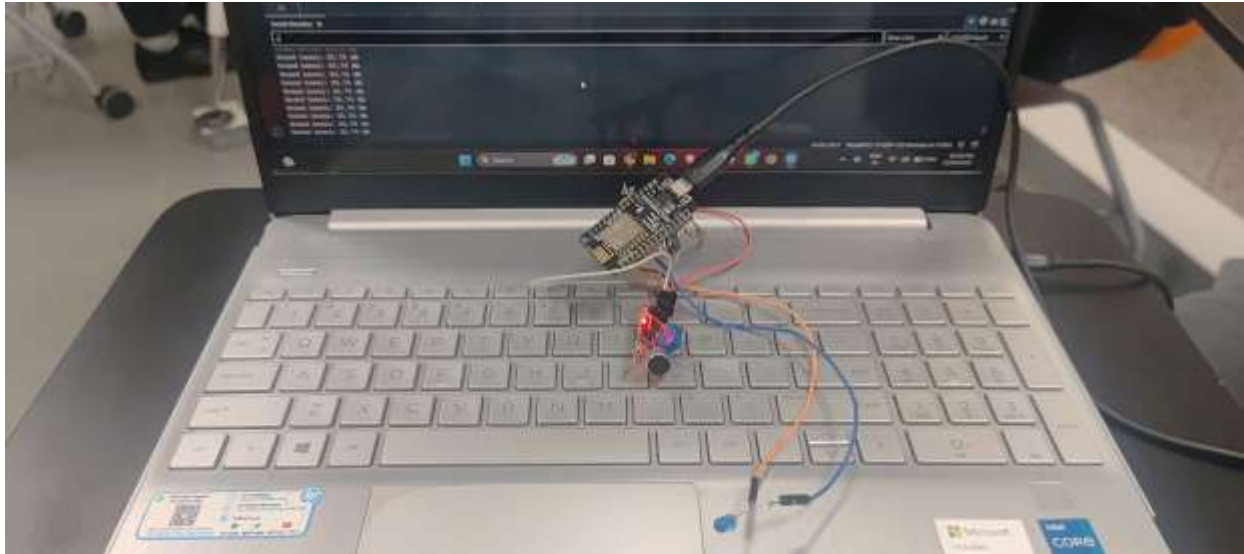


Figure 1: Assembled hardware setup — NodeMCU ESP8266 with sound sensor and LED, live serial output visible on screen

7. Firmware Overview

The firmware running on the NodeMCU is organised around the standard embedded control-loop pattern, consisting of an initialisation routine followed by a continuously repeating monitoring loop. Its logical flow is summarised below (implementation code is intentionally omitted from this report and is available separately as the project source file):

7.1 Initialisation

- Configure the serial communication interface for reporting readings.
- Set the LED pin as a digital output and ensure it starts in the OFF state.

7.2 Continuous Monitoring Loop

1. Begin a fixed-duration sampling window (50 ms).
2. Repeatedly read the analog sound sensor value within this window, tracking the maximum and minimum values observed.
3. At the end of the window, compute the peak-to-peak amplitude from the tracked maximum and minimum.
4. Convert the amplitude into a voltage value, then into an approximate decibel level using a logarithmic calculation.
5. Print the calculated decibel value to the serial monitor.
6. Compare the decibel value against the configured threshold (60 dB in the current implementation).
7. If the threshold is exceeded, switch the LED on and print an alert message; otherwise, keep the LED off.
8. Pause briefly before repeating the cycle, keeping the reporting rate steady and readable.

This structure keeps the sensing, processing, and actuation stages tightly coupled within a single loop iteration, ensuring the LED response and the reported reading always correspond to the same measurement cycle.

8. Results and Observations

The system was tested on the assembled hardware with the Arduino IDE Serial Monitor open to observe live readings. Under normal, quiet ambient conditions, the system consistently reported low and stable decibel values, with the LED remaining off. When a loud sound (such as a clap or spoken word near the microphone) was introduced, the reported dB value rose sharply, and once it crossed the configured 60 dB threshold, the LED switched on immediately, accompanied by an alert message printed on the serial monitor.

Key observations from testing:

- The system responds to sound-level changes within a single 50 ms sampling window, giving a near-instantaneous visual alert.
- Readings in a quiet room remained low and relatively steady, confirming that the ambient noise floor was correctly captured.
- The LED indicator behaved reliably and consistently, switching on only when the threshold was genuinely exceeded and switching off again once sound levels dropped.
- Serial output provided a clear, continuously updating log, which was useful both for debugging the sensor's baseline readings and for tuning the threshold value.
- Because the sensor's analog output and the sampling approach are not laboratory-calibrated, the reported dB values are best treated as a relative, repeatable measure of sound intensity rather than an absolute acoustic reference.

A representative set of test readings captured during evaluation is shown below:

Test Condition	Approx. Sound Level / LED State
Silent room, no activity	~25 dB — LED OFF
Normal speech near microphone	~40–50 dB — LED OFF
Hand clap close to sensor	63–68 dB — LED ON, alert printed
Loud shout / sharp sound	70+ dB — LED ON, alert printed

9. Applications

The underlying design is directly applicable to a range of practical noise-monitoring scenarios, including:

- Library, classroom, or office noise-level monitoring with a visual quiet/loud indicator.
- Baby-room or hospital ward monitoring, alerting caretakers to sudden loud sounds.
- Industrial workspace noise alerts to support workplace safety and hearing-protection compliance.
- A foundational sensing node for a smart-home system, where the sound alert can later be combined with other automation triggers.
- An entry point for a Wi-Fi-connected IoT noise-logging dashboard, leveraging the ESP8266's built-in wireless capability.

10. Future Scope

The current implementation can be extended in several meaningful directions:

1. **Wi-Fi Connectivity:** Using the ESP8266's onboard Wi-Fi to push readings to a cloud platform or a custom web dashboard for remote, real-time monitoring.
2. **Data Logging:** Storing historical sound-level data (locally or in a database) to analyse noise trends over time.
3. **Mobile/App Alerts:** Sending push notifications or messages when the threshold is crossed, instead of relying solely on a local LED.
4. **Calibration:** Using a reference sound level meter to calibrate the dB calculation for more accurate, standardised readings.
5. **Multi-level Alerts:** Introducing multiple thresholds (e.g., moderate and severe) with different LED colours or buzzer patterns for graded responses.
6. **Integration with Smart-Home Systems:** Feeding sound-level events into a broader home-automation setup for context-aware actions.

11. Bill of Materials

The table below lists the components used in this project. Costs are approximate and can vary by vendor and region; the overall build remains inexpensive, which is one of the key advantages of this design.

Item	Approx. Quantity
NodeMCU ESP8266 development board	1
Analog sound sensor module	1
LED (5 mm, any colour)	1
Jumper wires (male-to-male / male-to-female)	As required
Micro-USB cable	1

12. Conclusion

This project successfully demonstrates a functional, low-cost sound level monitoring and alert system using the NodeMCU ESP8266 and an analog sound sensor. By sampling the sensor's output, computing an approximate decibel value, and comparing it against a threshold, the system reliably provides real-time visual alerts for elevated sound levels while continuously logging readings over a serial connection. The project reinforces key embedded systems concepts such as analog signal sampling, signal-to-decibel conversion, threshold-based control logic, and digital output actuation, while also establishing a clear and extensible foundation for a future Wi-Fi-connected IoT noise-monitoring solution.

13. Project Video



**Sound Sensor
Prj.Video.mp4**

Note: Double Click on above file to open Video.